C Programming

Assignment 2 (CO2) - Report

Operators, Decision-Making Statements and Looping Constructs

Questions 1 to 4: Precedence and Associativity, Short-Circuit Evaluation, Bitwise Operators, else-if Ladder vs switch

Student name	P.SAI SUSHANTH REDDY
Register number	1925260171
Course / Faculty	
GitHub repository	https://github.com/nagagowthamjanjanam-code/CSA02-1965260183-Assignment-2
Compiler used for measurements	GCC 13.3.0 (Ubuntu 24.04). Compile command: `gcc -Wall -Wextra -o qN qN.c`
Screenshots	Student's own execution screenshots from an online C compiler (Figures 1 to 5)

Fill in the name, register number, faculty and the GitHub link before submission. The assignment states that a report without a working GitHub link is not evaluated.

Contents and marks

Section	Topic	Marks
Q1	Precedence, associativity and the discount bug (K4)	6
Q2	When does short-circuit evaluation matter? (K4)	4
Q3	Device-status register with bitwise operators, Complete the Code (K4)	6
Q4	else-if ladder or switch? Complete the Code (K4)	6
Appendix	Repository layout, build commands and reproducibility	-

Questions 5 to 7 are not part of this document.

Method used in every section

Each question follows the order required by the assignment: **Predict, Design, Implement, Measure, Analyse, Evaluate**. Predictions were worked out by hand from the C rules before any program was run. Raw counts (function calls, comparisons, loop iterations) are the primary evidence; the screenshots only confirm the final output. Counts are deterministic and can be reproduced exactly; the few timing figures depend on the machine and are used only as supporting observations.

Q3 - Device-Status Register with Bitwise Operators

Smart-home controller. 6 marks, Bloom K4 (Analyse). Includes Complete the Code.

3.1 Problem analysis

One **unsigned char** holds eight device states, bit *n* for device *n* (LIGHT = bit 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4). A mask with only bit *dev* set is `1u << dev`. Each operation uses one property of a bitwise operator:

Operation	Expression	Why it works
Switch on	`reg | (1u << dev)`	x OR 1 = 1 forces the bit to 1; x OR 0 = x leaves the other bits unchanged
Switch off	`reg & ~(1u << dev)`	~mask has a 0 only at bit dev; x AND 0 = 0 clears it, x AND 1 = x keeps the others
Toggle	`reg ^ (1u << dev)`	x XOR 1 flips x; x XOR 0 = x
Test	`(reg & (1u << dev)) != 0`	AND with the mask keeps only bit dev; the result is non-zero exactly when it is 1
Print in binary	`((reg >> bit) & 1u)`	shift bit to position 0, then mask it
Count devices ON	`while (r) { r &= r - 1; n++; }`	r & (r - 1) clears the lowest set bit, so the loop runs once per ON device

3.2 Complete the Code

TODO 1 to 4 were completed using only bitwise operators; the rest of the program is unchanged.

```
#include <stdio.h>
/* Each bit of one byte stores the ON/OFF state of one device. */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
    return reg | (1u << dev);          /* TODO 1: set bit dev */
}
unsigned char switch_off(unsigned char reg, int dev)
{
    return reg & ~(1u << dev);         /* TODO 2: clear bit dev */
}
unsigned char toggle(unsigned char reg, int dev)
{
    return reg ^ (1u << dev);          /* TODO 3: flip bit dev */
}
int is_on(unsigned char reg, int dev)
{
    return (reg & (1u << dev)) != 0;   /* TODO 4: test bit dev */
}
void show(unsigned char reg)
{
    for (int bit = 7; bit >= 0; bit--) putchar(((reg >> bit) & 1u) ? '1' : '0');
    putchar('\n');
}
int main(void)
{
    unsigned char reg = 0;
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);
    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);
    printf("AC is %s, PUMP is %s\n", is_on(reg, AC) ? "ON" : "OFF",
           is_on(reg, PUMP) ? "ON" : "OFF");
    return 0;
}
```

Prediction: register value after each statement of main()

Statement	Register (binary)	Decimal	Printed by show()
unsigned char reg = 0;	00000000	0	-
reg = switch_on(reg, LIGHT);	00000001	1	-
reg = switch_on(reg, AC);	00000101	5	00000101
reg = toggle(reg, FAN);	00000111	7	-
reg = switch_off(reg, LIGHT);	00000110	6	00000110
is_on(reg, AC) (bit 2)	bit 2 = 1	-	AC is ON
is_on(reg, PUMP) (bit 4)	bit 4 = 0	-	PUMP is OFF

Observed output (program run)	Expected output given in the assignment	Match
00000101 00000110 AC is ON, PUMP is OFF	00000101 00000110 AC is ON, PUMP is OFF	Identical

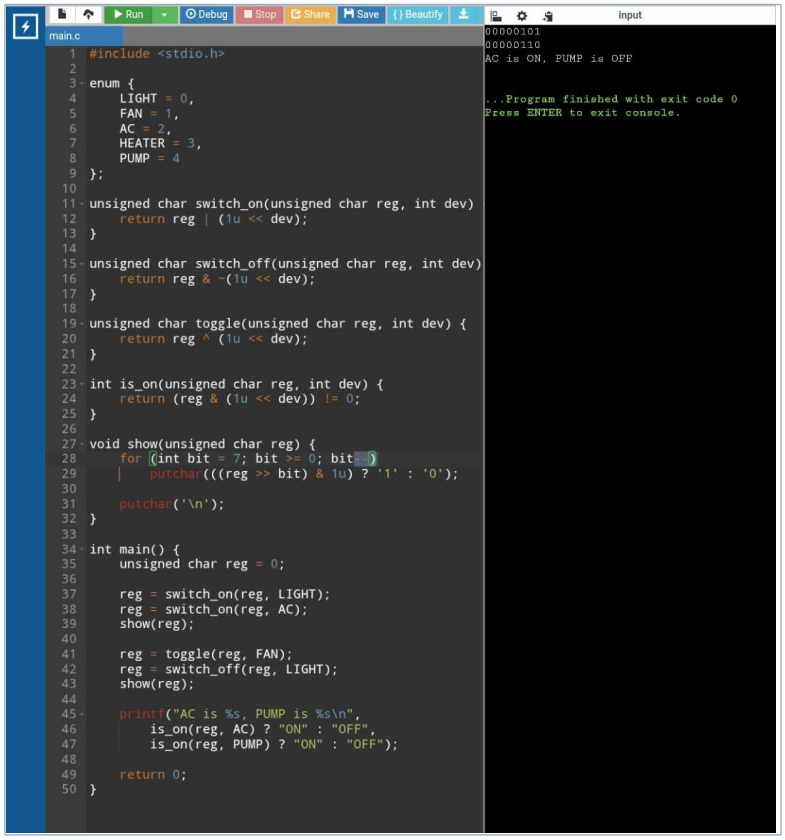


Figure 3 - Student's execution of the completed program: 00000101, 00000110, "AC is ON, PUMP is OFF".

3.3 My own predicted sequence and register trace

A second sequence was designed to include the boundary device (bit 7), an operation that changes nothing (clearing a bit that is already 0) and toggles. Each predicted value was written down first; the program q3_trace.c then compared them.

```
== Part 1: predicted vs observed register after each statement ==
statement              pred.bin  dec  obs.bin   dec  match
switch_on(HEATER)      00001000 8    00001000 8    yes
switch_on(PUMP)        00011000 24   00011000 24   yes
switch_on(FAN)         00011010 26   00011010 26   yes
toggle(HEATER)         00010010 18   00010010 18   yes
toggle(LIGHT)          00010011 19   00010011 19   yes
switch_off(PUMP)       00000011 3    00000011 3    yes
switch_off(AC)         00000011 3    00000011 3    yes
switch_on(device 7)    10000011 131  10000011 131  yes
devices ON in final register = 3 (loop iterations = 3)
```

All eight predictions matched. Note that switch_off(AC) on a bit that is already 0 leaves the register unchanged (idempotent), and that three devices are ON at the end (LIGHT, FAN, device 7). The Kernighan counting loop ran exactly 3 iterations for 3 ON devices.

3.4 Inquiry: precedence and signedness

reg & 1 << dev versus (reg & 1) << dev

In C, **<< has higher precedence than &**. So `reg & 1 << dev` means `reg & (1 << dev)`: it tests bit *dev*, which is what is wanted. In `(reg & 1) << dev` the brackets force the AND first: it extracts bit 0 only (the LIGHT bit) and then moves that result to position *dev*. It does not look at bit *dev* at all.

```
reg=00000100 (AC on): reg & 1 << 2 = 4 ; (reg & 1) << 2 = 0          r
eg=00000001 (LIGHT on, AC off): reg & 1 << 2 = 0 ; (reg & 1) << 2 = 4
```

With only AC on, the correct form gives 4 (non-zero, AC is on) but the wrong form gives 0 (AC looks off). With only LIGHT on, the wrong form gives 4 and falsely reports AC as on. A related trap: `reg & mask == 0` parses as `reg & (mask == 0)` because == binds tighter than &, so the test must be written `(reg & mask) != 0`.

What goes wrong with signed char when bit 7 is set

```
signed char : value=-128  s>>7=-1  (s>>7)==1:0  s==0x80:0  s<0:1  u
nsigned     : value=128  u>>7=1  (u>>7)==1:1  u==0x80:1  u<0:0   b
it test (s & (1<<7))!=0 -> 1 ; ~s as int = 127 ; ~u as int = -129(u
nsigned char)~u = 127
```

- Setting bit 7 makes a signed char **negative** (observed value -128, and `s < 0` is true). Converting 128 into signed char is implementation-defined.
- When a char is used in an expression it is promoted to int with **sign extension**. The register compares as -128, so `s == 0x80` is false (0x80 is 128) while the unsigned version gives true.
- Right shift of a negative value is implementation-defined; GCC does an **arithmetic shift**, filling with 1s, so `s >> 7` is -1, not 1, and `(s >> 7) == 1` is false.
- Printing the register as a number shows -128 instead of 128. The mask test `(s & (1 << 7)) != 0` still works, which hides the problem until another operation is used.
- Even for unsigned char, integer promotion applies: `~u` is the int -129, not an 8-bit value. The functions return unsigned char, so the result is truncated back to 8 bits, but intermediate results printed or compared must be cast: `(unsigned char)~u = 127`. **Rule: keep the register unsigned (unsigned char or uint8_t) and use 1u masks.**

3.5 Alternative concept: char state[8] (one byte per device)

```
void on(char s[8], int d)     { s[d] = 1; }
void off(char s[8], int d)    { s[d] = 0; }
void toggle(char s[8], int d) { s[d] ^= 1; }
int  is_on(char s[8], int d)  { return s[d]; }
int  count(char s[8])         { int n = 0; for (int i = 0; i < 8; i++) n += s[i]; return n; }
```

```
sizeof(unsigned char)=1 byte, sizeof(HomeArr)=8 bytes
1,000,000 homes: bit-register = 1000000 bytes, array = 8000000 bytes
count devices ON, 1,000,000 homes x 20 repeats (totals must agree: 79939780 79939780 79939780)
array scan     : 160000000 loop iterations per pass-set, 0.085 s
bit (Kernighan): 79939780 loop iterations per pass-set, 0.198 s b
it + 256-table: 20000000 lookups per pass-set,            0.008 s
```

Measure	One bit per device (unsigned char)	Array char state[8]
Time: set, clear, toggle, test	O(1): one shift plus one bitwise operation	O(1): one indexed access
Space per home	1 byte	8 bytes (sizeof = 8, measured)
Space for 1,000,000 homes	1,000,000 bytes (about 1.0 MB)	8,000,000 bytes (about 8.0 MB); 7 MB more
Count devices ON, per home	Kernighan loop: one iteration per ON bit; average 4.00 over all 256 register values (measured total 1024); a 256-entry table gives 1 lookup	always 8 iterations (2048 over all 256 states)
Count, 1,000,000 homes x 20 passes	79,939,780 iterations, 0.198 s (table version: 20,000,000 lookups, 0.008 s)	160,000,000 iterations, 0.085 s
Totals agree?	yes (79,939,780)	yes (79,939,780)

Analysis. Per-device operations are O(1) in both designs, and counting the ON devices of one home is O(8) = O(1) in both; over H homes counting is O(H). The bit register uses one eighth of the memory, which matters on a controller with a few bytes of RAM and when a million states are stored or sent over a network. The array is easier to read and extends beyond 8 devices without changing the type. Counting is the interesting case: the Kernighan loop does half the iterations of the array scan (4.0 on average versus 8) but was *slower* in wall-clock time here (0.198 s versus 0.085 s at -O2), probably because its loop length depends

on the data and the branch is hard to predict, whereas the array loop has a fixed length. The 256-entry lookup table (256 extra bytes) was fastest by a wide margin (0.008 s). Timings are machine-specific; the iteration counts are exact. The bit register also allows whole-register operations in one instruction (switch everything off with reg = 0, compare two states with ^).

Source code

```
#include <stdio.h>
#include <assert.h>
/* Q3: register trace with predictions, popcount, precedence and signedness experiments */
enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4, DEV7 = 7 };
unsigned char switch_on(unsigned char r, int d)  { return r | (1u << d); }
unsigned char switch_off(unsigned char r, int d) { return r & ~(1u << d); }
unsigned char toggle(unsigned char r, int d)     { return r ^ (1u << d); }
int is_on(unsigned char r, int d)                { return (r & (1u << d)) != 0; }
void bin(unsigned char r) { for (int b = 7; b >= 0; b--) putchar(((r >> b) & 1u) ? '1' : '0'); }

int count_on(unsigned char r, int *iters)   /* Kernighan: each pass clears the lowest set bit */
{
    int n = 0; *iters = 0;
    while (r) { r &= r - 1; n++; (*iters)++; }
    return n;
}

int main(void)
{
    printf("== Part 1: predicted vs observed register after each statement ==\n");
    struct { const char *stmt; int op, dev; unsigned char predicted; } steps[] = {
        {"switch_on(HEATER)",  0, HEATER, 0x08}, {"switch_on(PUMP)",     0, PUMP,   0x18},
        {"switch_on(FAN)",     0, FAN,    0x1A}, {"toggle(HEATER)",      2, HEATER, 0x12},
        {"toggle(LIGHT)",      2, LIGHT,  0x13}, {"switch_off(PUMP)",    1, PUMP,   0x03},
        {"switch_off(AC)",     1, AC,     0x03}, {"switch_on(device 7)",0, DEV7,   0x83},
    };
    unsigned char reg = 0;
    printf("%-22s %-9s %-4s %-9s %-4s %s\n", "statement", "pred.bin", "dec", "obs.bin", "dec", "match");
    for (int i = 0; i < 8; i++) {
        unsigned char before = reg;
        reg = steps[i].op == 0 ? switch_on(reg, steps[i].dev)
            : steps[i].op == 1 ? switch_off(reg, steps[i].dev) : toggle(reg, steps[i].dev);
        (void)before;
        printf("%-22s ", steps[i].stmt); bin(steps[i].predicted); printf(" %-4d ", steps[i].predicted);
        bin(reg); printf(" %-4d %s\n", reg, reg == steps[i].predicted ? "yes" : "NO");
    }
    int it; int n = count_on(reg, &it);
    printf("devices ON in final register = %d (loop iterations = %d)\n", n, it);

    printf("\n== Part 2: reg & 1 << dev  versus  (reg & 1) << dev ==\n");
    unsigned char t1 = 0x04, t2 = 0x01; int dev = 2;
    printf("reg=00000100 (AC on): reg & 1 << 2 = %d ; (reg & 1) << 2 = %d\n", t1 & 1 << dev, (t1 & 1) << dev);
    printf("reg=00000001 (LIGHT on, AC off): reg & 1 << 2 = %d ; (reg & 1) << 2 = %d\n", t2 & 1 << dev, (t2 & 1) << dev);

    printf("\n== Part 3: signed char with bit 7 set ==\n");
    signed char s = 0; s = s | (1 << 7);
    unsigned char u = 0; u = u | (1 << 7);
    printf("signed char : value=%d  s>>7=%d  (s>>7)==1:%d  s==0x80:%d  s<0:%d\n", s, s >> 7, (s >> 7) == 1, s == 0x80, s < 0);p
    rintf("unsigned     : value=%d  u>>7=%d  (u>>7)==1:%d  u==0x80:%d  u<0:%d\n", u, u >> 7, (u >> 7) == 1, u == 0x80, u < 0);p
    rintf("bit test (s & (1<<7))!=0 -> %d ; ~s as int = %d ; ~u as int = %d\n", (s & (1 << 7)) != 0, ~s, ~u);                pri
    ntf("(unsigned char)~u = %d\n", (unsigned char)~u);

    printf("\n== Part 4: count-ON cost over all 256 register values ==\n");
    long tot_k = 0;
    for (int v = 0; v < 256; v++) { int i2; count_on((unsigned char)v, &i2); tot_k += i2; }
    printf("Kernighan total iterations = %ld (average %.2f); array scan = 8 per register (%d total)\n", tot_k, tot_k / 256.0, 256 * 8);
    return 0;
}
```

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
/* Q3 alternative: one byte per device (char state[8]) versus one bit per device (unsigned char) */
#define HOMES 1000000
typedef struct { char state[8]; } HomeArr;

int main(void)
{
    printf("sizeof(unsigned char)=%zu byte, sizeof(HomeArr)=%zu bytes\n", sizeof(unsigned char), sizeof(HomeArr));
    printf("1,000,000 homes: bit-register = %zu bytes, array = %zu bytes\n",
           sizeof(unsigned char) * (size_t)HOMES, sizeof(HomeArr) * (size_t)HOMES);

    unsigned char *regs = malloc(HOMES);
    HomeArr *arr = malloc(sizeof(HomeArr) * HOMES);
    unsigned int s = 2463534242u;
    for (int i = 0; i < HOMES; i++) {
        s ^= s << 13; s ^= s >> 17; s ^= s << 5;
        regs[i] = (unsigned char)s;
        for (int d = 0; d < 8; d++) arr[i].state[d] = (regs[i] >> d) & 1;
    }
    unsigned char popc[256];
    for (int v = 0; v < 256; v++) { int c = 0; for (int b = 0; b < 8; b++) c += (v >> b) & 1; popc[v] = (unsigned char)c; }

    long total_a = 0, total_k = 0, total_t = 0, it_a = 0, it_k = 0;
    clock_t t0 = clock();
    for (int r = 0; r < 20; r++) for (int i = 0; i < HOMES; i++) { for (int d = 0; d < 8; d++) { total_a += arr[i].state[d]; it_a++; } }
```

```
    clock_t t1 = clock();
    for (int r = 0; r < 20; r++) for (int i = 0; i < HOMES; i++) { unsigned char v = regs[i]; while (v) { v &= v - 1; total_k++; it_k++; } }
    clock_t t2 = clock();
    for (int r = 0; r < 20; r++) for (int i = 0; i < HOMES; i++) total_t += popc[regs[i]];
    clock_t t3 = clock();
    printf("count devices ON, 1,000,000 homes x 20 repeats (totals must agree: %ld %ld %ld)\n", total_a, total_k, total_t);
    printf("array scan     : %ld loop iterations per pass-set, %.3f s\n", it_a, (double)(t1 - t0) / CLOCKS_PER_SEC);
    printf("bit (Kernighan): %ld loop iterations per pass-set, %.3f s\n", it_k, (double)(t2 - t1) / CLOCKS_PER_SEC);
    printf("bit + 256-table: %d lookups per pass-set,           %.3f s\n", 20 * HOMES, (double)(t3 - t2) / CLOCKS_PER_SEC);
    free(regs); free(arr);
    return 0;
}
```